

Table des matières

Aide-mémoire — les concepts de base..... 1

1. La grande image : de la donnée brute au tableau de bord..... 1

2. Snowflake : les 3 questions qu’on se pose toujours 4

3. L’arborescence des objets Snowflake 4

4. Le lexique du code SQL (ce que vous tapez dans un worksheet) 5

5. Glossaire des concepts..... 6

6. Les fichiers d’un projet dbt 9

Comment utiliser cette fiche 11

Aide-mémoire — les concepts de base

Formation Analytics Engineering — à garder ouvert à côté des cours

À quoi sert cette fiche. Une vue très générale pour situer tout ce que la formation manipule : le pipeline (où on va), les objets Snowflake (rôle, entrepôt, base...), et le vocabulaire du code SQL. Aucun détail ici, juste de quoi ne jamais être perdu. Revenez-y dès qu’un mot vous bloque. Le fil rouge, rappelez-vous, c’est Boréal Powersports : un fabricant de véhicules récréatifs dont on récupère les extraits type SAP.

1. La grande image : de la donnée brute au tableau de bord

Toute la formation construit une seule chaîne. On part des extraits SAP de Boréal (des fichiers), on les range et on les transforme par étapes dans Snowflake, puis on les montre dans Power BI. Le trajet, en une ligne :

Fichiers SAP (CSV) → RAW → STAGING → INTERMEDIATE → MARTS (étoile) → PRESENTATION → Power BI, le tout versionné avec Git et testé / déployé en CI/CD. Le socle (RAW, STAGING, MARTS) est présent partout ; INTERMEDIATE et PRESENTATION s’ajoutent au besoin (Boréal les pose). Toutes ces couches vivent dans Snowflake, qui stocke et calcule ; Power BI ne fait que restituer.

Couche	Ce qu’on y fait	Statut	Avec quel outil
Source	les extraits SAP de départ (5 fichiers CSV)	—	fichiers (chargés au Cours 01)

Couche	Ce qu'on y fait	Statut	Avec quel outil
RAW	copie brute, on n'y touche pas	socle	SQL — stage + COPY INTO (Snowflake)
STAGING	nettoyage 1 pour 1 (renommer, typer, décoder)	socle	dbt
INTERMEDIATE	assembler / calculer une logique réutilisable (avant les marts)	au besoin	dbt
MARTS	modèle en étoile, prêt à analyser	socle	dbt
PRESENTATION	tables pré-agrégées, prêtes pour un rapport	au besoin	dbt
Restitution	rapports, tableaux de bord	socle	Power BI
(transverse)	versionner, tester, déployer	—	Git + GitHub Actions

Différence avec une source « déjà là ». Contrairement à des données d'exemple déjà présentes dans l'entrepôt, les extraits SAP de Boréal sont de vrais fichiers qu'on doit faire entrer (Cours 01, stage + COPY INTO). C'est plus proche du réel, et ça donne au staging un vrai travail (dates en texte, codes à décoder).

Conçue à un cours, bâtie à un autre. Le point qui embrouille : *concevoir* une couche (décider sa forme) et la *bâtir* (l'écrire) n'arrivent pas au même cours, et seuls les MARTS se conçoivent. Le staging, lui, n'a rien à décider : c'est mécanique (une vue par table).

Couche	Conçue (on décide sa forme)	Bâtie (on l'écrit)
RAW	— (copie telle quelle)	Cours 01
STAGING	— (rien à décider : 1 vue de nettoyage par table)	Cours 03
INTERMEDIATE	— (mécanique : logique réutilisable)	Cours 04
MARTS	Cours 02 (l'étoile Kimball)	Cours 04
PRESENTATION	— (mécanique : agrégats de rapport)	Cours 06

Autrement dit : seuls les MARTS se conçoivent (Cours 02 dessine l'étoile) ; les autres couches ne s'inventent pas, elles s'intercalent (staging pour nettoyer, intermediate pour assembler, presentation pour résumer) et se bâtissent directement.

Les deux couches que dbt bâtit : STAGING contre MARTS

C'est le point qui embrouille entre les Cours 03 et 04. Les deux sont des modèles dbt (un .sql = un SELECT, reliés dans le DAG, versionnés et testés), mais ils ne font pas le même métier :

	STAGING (Cours 03)	MARTS (Cours 04)
Son métier	nettoyer le brut (<i>renommer, typer, décoder</i>)	modéliser (l'étoile)
Une table source...	...donne 1 vue (1 pour 1)	...se combine aux autres (jointures)
Ce qu'il lit	source() (le brut RAW)	ref() (le staging)
Materialization	view (rien de stocké)	table (figée sur disque)
Jointures, logique métier	non, jamais	oui — c'est là qu'on assemble
Clés de substitution	non	oui (dim + fait)
Nom des modèles	stg_<source> (<i>stg_vbap...</i>)	dim_* / fct_ventes
Ce qu'on obtient	une façade propre sur le brut	le modèle prêt à analyser

En une phrase : STAGING range le brut, table par table ; MARTS l'assemble en étoile. Même outil, deux étages. On nettoie d'abord, on modélise ensuite.

La raison d'un découpage en deux cours. Le Cours 03 vous fait découvrir dbt sur la couche la plus simple (STAGING : des vues 1:1, aucune jointure), l'endroit idéal pour apprendre la mécanique de l'outil. Le Cours 04 réutilise ces bases sur la couche riche (MARTS : jointures, clés, tables) et ajoute ce qui fait un livrable pro (tests, historique, doc). Les « fondations de dbt » ne sont donc pas un chapitre à part : ce sont les bases apprises au Cours 03 en bâtissant STAGING.

Concrètement, dans Snowsight

Ce ne sont pas des idées abstraites : STAGING et MARTS sont deux tiroirs (schémas) dans la base BOREAL, que dbt remplit. Ouvrez Snowsight, dépliez BOREAL, et regardez dedans :

Ce que vous ouvrez	Ce que vous y trouvez	Un SELECT * donne...
BOREAL . STAGING	des vues STG_KNA1, STG_VBAP... (une par table du brut)	les mêmes lignes que le brut, mais propres : colonnes renommées, dates typées, palier décodé. Rien de neuf — juste lisible.
BOREAL . MARTS	des tables DIM_PRODUIT, DIM_CONCESSIONNAIRE, FCT_VENTES	l'étoile : une ligne par vente, déjà reliée à ses dimensions, marge calculée. C'est ça que Power BI lira.

Donc : `SELECT * FROM BOREAL.STAGING.STG_KNA1` → un concessionnaire par ligne, colonnes claires ; `SELECT * FROM BOREAL.MARTS.FCT_VENTES` → le fait des ventes, prêt à analyser. Vous pouvez les voir et les interroger, ce sont de vrais objets, pas des concepts.

L'image de la cuisine. RAW = les caisses de livraison telles qu'elles arrivent (légumes en vrac, non lavés, étiquettes cryptiques). STAGING = la mise en place : chaque ingrédient lavé, épluché, coupé, dans son propre bac étiqueté (un bac par ingrédient, on ne mélange pas encore). MARTS = le plat dressé : les ingrédients assemblés selon la recette (l'étoile), prêts à servir (à Power BI). On prépare d'abord (staging), on assemble ensuite (marts). Deux gestes distincts, deux tiroirs distincts.

Un même type de couche, plusieurs schémas. Les couches ci-dessus sont des types de rôle, pas des tiroirs uniques. Deux choses à retenir :

- INTERMEDIATE et PRESENTATION ne sont pas exotiques : ce sont des couches standard que Boréal pose (intermediate au Cours 04, presentation au Cours 06). Elles s'ajoutent au socle dès qu'il y a de la logique réutilisable (`int_*`) ou un rapport à pré-agréger (`rpt_*`).
- Un type peut se dédoubler en plusieurs schémas : plusieurs `raw_*/staging_*` (une par source), plusieurs `marts_*/presentation_*` (un par domaine métier), reliés par des dimensions conformes. C'est le cas du projet synthèse Aurore (`multi_raw`, `multi_marts...`).

Principe : on structure autant que le projet l'exige, pas plus.

2. Snowflake : les 3 questions qu'on se pose toujours

C'est la source de confusion, alors on la règle une fois pour toutes. Avant d'exécuter quoi que ce soit, Snowflake veut 3 réponses :

Question	Objet	C'est...	On le choisit avec
Qui suis-je ? (mes droits)	ROLE (rôle)	votre badge de permissions	USE ROLE ...
Quel moteur calcule ?	WAREHOUSE (entrepôt)	le moteur qui exécute, payé à l'usage	USE WAREHOUSE ...
Où sont les données ?	DATABASE.SCHEMA	l'adresse des tables	USE DATABASE ... puis USE SCHEMA ...

Analogie simple : - le ROLE = votre carte d'accès (ce que vous avez le droit de faire) ; - le WAREHOUSE = le moteur / la machine qui fait le travail ; - la DATABASE.SCHEMA = le classeur → tiroir où sont rangées les tables.

Les trois sont indépendants : les mêmes données peuvent être lues avec des rôles ou des moteurs différents. Dans nos labos, on fixe toujours le trio en tête de requête : `USE ROLE BOREAL_ROLE; USE WAREHOUSE BOREAL_WH; USE DATABASE BOREAL;`

3. L'arborescence des objets Snowflake

Un compte Snowflake range ses objets sur plusieurs branches séparées, qu'on combine au moment de travailler. Les données, elles, sont tout au fond, en descendant base → schéma → table (un point par niveau : BOREAL . RAW . VBAP).

Niveau	Ce que c'est	Chez Boréal
Compte	le conteneur de tout	votre compte d'essai
Utilisateurs	qui se connecte	vous : votre identifiant Snowflake
Rôles	badges de droits	BOREAL_ROLE, SYSADMIN, ACCOUNTADMIN...
Entrepôts (<i>warehouses</i>)	moteurs de calcul	BOREAL_WH
Bases de données	où vivent les données	BOREAL
↳ Schémas	des « tiroirs » dans la base	RAW, STAGING, MARTS, DEV (<i>un projet complexe en ajoute : INTERMEDIATE, MARTS_<domaine>, PRESENTATION...</i>)
↳ Tables / vues	les données elles-mêmes	VBAP, VBAK, KNA1, MARA, T005S...

À retenir : Rôles, Entrepôts et Bases sont 3 branches séparées du compte, on les combine au moment de travailler (le trio de la section 2). Les données, elles, se trouvent en descendant base → schéma → table.

4. Le lexique du code SQL (ce que vous tapez dans un worksheet)

Mot-clé	Ce qu'il fait	Mini-exemple
USE ...	choisit le contexte (rôle / entrepôt / base / schéma)	USE ROLE BOREAL_ROLE;
CREATE ...	crée un objet	CREATE DATABASE BOREAL;
CREATE OR REPLACE ...	crée, ou remplace s'il existe (rejouable)	CREATE OR REPLACE TABLE ...
IF NOT EXISTS	« ne plante pas si ça existe déjà »	CREATE SCHEMA IF NOT EXISTS RAW;
GRANT <droit> ON <objet> TO <qui>	accorde une permission	GRANT USAGE ON WAREHOUSE BOREAL_WH TO ROLE BOREAL_ROLE;

Mot-clé	Ce qu'il fait	Mini-exemple
SELECT ... FROM ...	lit des données	SELECT * FROM BOREAL.RAW.VBAP;
CREATE TABLE X AS SELECT ... (CTAS)	crée une table et la remplit d'un coup	CREATE TABLE T AS SELECT * FROM ...;
COPY INTO ... FROM @stage	charge un fichier d'un stage dans une table	COPY INTO VBAP FROM @STAGE_SAP/VBAP.csv;
SHOW <objets>	liste des objets	SHOW TABLES IN SCHEMA BOREAL.RAW;
COUNT(*)	compte les lignes	SELECT COUNT(*) FROM BOREAL.RAW.VBAP;
;	termine une instruction	chaque commande finit par ;
--	un commentaire (ignoré à l'exécution)	-- ceci est une note

Principes de lecture du code : - Chaque instruction se termine par ;. Pour exécuter tout un bloc d'un coup dans Snowsight, on utilise Run All (pas juste *Run*, qui n'exécute qu'une ligne). - Le nom complet d'une table s'écrit BASE.SCHÉMA.TABLE (ex. BOREAL.RAW.VBAP). Si vous avez déjà fait USE DATABASE / USE SCHEMA, vous pouvez écrire juste VBAP. - Snowflake ignore la casse des noms d'objets ; par convention on les écrit en MAJUSCULES.

5. Glossaire des concepts

Terme	En une phrase
Entrepôt de données (<i>data warehouse</i>)	une base pensée pour analyser de gros volumes (≠ base d'une appli). Snowflake en est un.
OLTP vs OLAP	OLTP = base d'application (transactions du quotidien, ex. l'ERP SAP) ; OLAP = analyse (rapports). Nous faisons de l'OLAP.
Snowflake	l'entrepôt cloud où vivent et se calculent nos données.
Rôle (<i>role</i>)	un badge de permissions, attribué à des utilisateurs. (RBAC = gérer les droits par rôles.)
Entrepôt virtuel (<i>warehouse</i>)	le moteur de calcul de Snowflake, payé à la seconde quand il tourne.
Base / schéma / table	base = grand conteneur ; schéma = tiroir ; table = les lignes/colonnes de données.

Terme	En une phrase
Vue (<i>view</i>)	une requête enregistrée qui s'utilise comme une table (mais ne stocke rien).
RAW / STAGING / MARTS	les 3 couches : brut → nettoyé → modèle prêt à analyser. La colonne vertébrale.
Couche INTERMEDIATE (<i>int_*</i>)	couche facultative entre staging et marts : de la logique réutilisable par plusieurs marts, souvent ephemeral.
Couche PRESENTATION (<i>rpt_*</i>)	couche facultative au-dessus des marts : des tables pré-agrégées taillées pour un rapport.
Data mart / plusieurs marts	un projet à plusieurs processus découpe ses marts par domaine (MARTS_VENTES, MARTS_LOGISTIQUE...) : plusieurs schémas de marts.
Dimensions conformes	une même dimension (<i>dim_date</i> , <i>dim_produit</i> ...) définie une fois et partagée par plusieurs faits/marts — ce qui les empêche de diverger.
Stage	un quai de déchargement où les fichiers attendent avant d'entrer dans une table (<i>COPY INTO</i>). Objet Snowflake, aucun rapport avec la couche STAGING : le stage tient des fichiers en amont de RAW, la couche tient des vues <i>stg_*</i> en aval (Théorie 01, section 5).
ELT	<i>Extract-Load-Transform</i> : on charge le brut, puis on transforme dans l'entrepôt.
dbt	un logiciel (programme en ligne de commande, <i>dbt run/dbt build</i>) — pas un langage ni une base : vous lui donnez du SQL dans des fichiers, il le compile et l'envoie à Snowflake. C'est le « T » (transformations) de l'ELT, avec tests et doc.
Modèle (<i>model</i> , dbt)	un fichier <i>.sql</i> = un <i>SELECT</i> qui devient une table ou une vue.
Materialization (dbt)	comment dbt matérialise un modèle : <i>view</i> (rien de stocké), <i>table</i> (figée), <i>incremental</i> , <i>ephemeral</i> .
DAG (<i>graphe de dépendances</i>)	l'ordre d'exécution que dbt déduit des <i>ref()</i> (qui dépend de qui) ; il construit de la source aux marts, tout seul.

Terme	En une phrase
Snapshot (dbt)	le mécanisme qui historise une dimension (SCD2) : il garde les anciennes versions au lieu de les écraser.
Étoile / fait / dimension	organisation pour l'analyse : une table de faits (les événements chiffrés, ex. ventes) entourée de dimensions (le contexte : concessionnaire, produit, date...).
Clé de substitution (<i>surrogate key</i>)	un identifiant technique (entier ou hash) inventé par l'entrepôt pour une ligne de dimension — c'est elle que le fait référence (≠ clé métier de la source, ex. KUNNR).
CTAS	CREATE TABLE AS SELECT : créer une table et la remplir en une commande.
Transient	table/schéma sans sauvegarde longue = moins cher, pour des données rejouables.
Power BI	l'outil qui transforme les tables finales en rapports / graphiques.
Import vs DirectQuery	Import copie les tables dans le fichier .pbix (rapide, fraîcheur = dernière actualisation) ; DirectQuery interroge Snowflake à chaque clic (frais, plus lent, plus coûteux). Import par défaut.
Modèle sémantique	le plan des tables et des relations à l'intérieur de Power BI. C'est l'étoile, redessinée dans l'outil : relations un-vers-plusieurs de la dimension vers le fait, filtrage à sens unique.
DAX	le langage de calcul de Power BI. Une mesure s'évalue à l'affichage, dans le contexte de filtre ; une colonne calculée se fige au chargement. On préfère les mesures.
Contexte de filtre	l'ensemble des filtres actifs au moment où une mesure s'évalue (l'année choisie, la région cliquée). C'est lui qui rend un rapport interactif.
CI / CD	intégration continue = rejouer automatiquement les vérifications à chaque proposition de changement ; déploiement continu = pousser en production ce qui a passé. Les deux sont indépendants.

Terme	En une phrase
GitHub Actions	l'exécuteur de la CI : un fichier <code>.yaml</code> décrit quoi lancer et quand, sur une machine neuve à chaque fois.
sqlfluff	le linter SQL : il vérifie la forme (casse, indentation, alias) et comprend la Jinja de dbt. Règle le style une fois pour toutes.
Fraîcheur (dbt source freshness)	contrôle l'ÂGE d'une source, pas son contenu. Le seul test qui attrape une table figée pendant que tous les autres restent verts.
manifest.json	le catalogue que dbt écrit à chaque exécution : modèles, colonnes, dépendances, tests, descriptions. C'est ce qui rend un projet dbt lisible par un outil — ou par un agent d'IA.
Git / GitHub Actions	versionner le code ; automatiser tests et déploiement (CI/CD).

6. Les fichiers d'un projet dbt

Un seul `dbt_project.yml` par projet, à sa racine. Tout le reste se range dans les dossiers que `dbt init` a créés.

Fichier	Où	Ce qu'il fait
<code>dbt_project.yml</code>	racine du projet	règle le projet : où écrire, en quoi matérialiser. Un seul.
<code>packages.yml</code>	racine du projet	les paquets externes (<code>dbt_utils</code>), installés par dbt deps
<code>profiles.yml</code>	<code>~/ .dbt/</code> , hors du dépôt	la connexion Snowflake (compte, rôle, mot de passe en <code>env_var</code>)
un modèle <code>.sql</code>	<code>models/...</code>	un <code>SELECT</code> que dbt enveloppe en <code>CREATE VIEW</code> ou <code>CREATE TABLE</code>
<code>_sources.yml</code>	<code>models/staging/</code>	déclare le brut que dbt n'a pas créé — c'est la base de <code>source()</code>
<code>_marts.yml</code>	<code>models/marts/</code>	descriptions et tests des modèles du dossier

Fichier	Où	Ce qu'il fait
une macro .sql	macros/	une fonction Jinja réutilisable (generate_schema_name)
un seed .csv	seeds/	du brut figé et versionné, chargé par dbt seed, appelé avec ref()

L'anatomie du dbt_project.yml

Deux parties : l'en-tête écrit par dbt init (on n'y touche pas) et les blocs de ressources que vous ajoutez cours après cours.

Partie	Clés	On y touche ?
en-tête — identité	name, version, profile	non
en-tête — chemins	model-paths, seed-paths, macro-paths, snapshot-paths, test-paths, analysis-paths, clean-targets	non
blocs de ressources	models:, seeds:, snapshots:	oui, à chaque couche ajoutée

Un bloc de ressources se lit sur quatre niveaux, et à chaque niveau le mot vient d'ailleurs :

```
models:           # 1. type de ressource – mot de dbt
  boreal:         # 2. le projet – le même que le `name:` de l'en-tête
    staging:      # 3. votre sous-dossier de models/
      +materialized: view # 4. un réglage – mot de dbt, toujours précédé d'un +
      +schema: staging
```

- Le + distingue un réglage d'un nom de dossier. Oublié, le réglage est ignoré sans message.
- Le réglage le plus précis gagne : dbt_project.yml (un dossier) < un .yml de description (un modèle) < {{ config(...) }} dans le .sql (ce modèle seul).
- On ne déclare que ce qu'on a : pas de seed, pas de bloc seeds: ; pas de couche intermédiaire, pas de ligne intermediate:. C'est pourquoi les trois projets de la formation n'ont pas le même fichier.
- Un modèle ephemeral ne crée aucun objet : il n'a pas besoin de +schema.

Qui a fabriqué la table de départ ? C'est la question qui décide de tout le reste :

Le brut est...	Chargé par	Réglé dans	Appelé avec
un seed (CSV versionné)	dbt seed	le bloc seeds: de dbt_project.yml	ref('client')
une source (chargée hors dbt)	COPY INTO, un ETL	un _sources.yml sous models/	source('raw', 'vbap')
un modèle	dbt run	le bloc models:	ref('stg_client')

Comment utiliser cette fiche

- Bloqué sur un mot du code → section 4.
- Bloqué sur un concept → section 5.
- Confusion rôle vs entrepôt vs base → section 2 et section 3.
- Perdu sur « où on va » → section 1.
- Un fichier du projet dbt vous surprend → section 6.